

Karlybay Diyar LAB1

System description

The system keeps the academic records of a university. The university consists of departments. Each department employs teachers, is responsible for a set of courses and forms study groups, and every student is placed in one of these groups. For every academic year and semester the university plans which course is taught by which teacher to which group. A student registers for such a taught course and receives a final result for it. This description is the source for all the entities, attributes and relationships listed below.

Task 1. Identified entities

Seven entities were taken from the description:

department, an academic unit of the university that employs teachers and owns courses;
teacher, an employee of a department who teaches courses;
course, a subject of study that belongs to a department;
study_group, a group of students formed by a department in one year of admission;
student, a person who studies in a group;
course_offering, a course taught by a teacher to a group in a given year and semester;
enrollment, the registration of a student for a taught course together with the result.

The last entity appeared because a student studies many courses and a course is studied by many students. This many-to-many link cannot be stored directly, so it is kept in a separate table.

Task 2. Attributes, keys and data types

department

department_id, integer, primary key
name, varchar(100), required, unique
head_name, varchar(100)
building, varchar(30)
phone, varchar(20)

teacher

teacher_id, integer, primary key
department_id, integer, foreign key to department, required
first_name, varchar(50), required
last_name, varchar(50), required
position, varchar(40), required (assistant, senior lecturer, associate professor, professor)
email, varchar(100), required, unique
hire_date, date, required

course

course_id, integer, primary key
department_id, integer, foreign key to department, required
course_code, varchar(10), required, unique
title, varchar(150), required
credits, smallint, required, value from 1 to 10
assessment_form, varchar(20), required (exam, credit, project)

study_group

group_id, integer, primary key
department_id, integer, foreign key to department, required
group_code, varchar(15), required, unique
enrollment_year, smallint, required
study_form, varchar(15), required (full-time, part-time, distance)

student

student_id, integer, primary key
group_id, integer, foreign key to study_group, required
first_name, varchar(50), required
last_name, varchar(50), required
date_of_birth, date, required
email, varchar(100), required, unique
phone, varchar(20)
status, varchar(15), required (active, academic leave, expelled, graduated)

course_offering

offering_id, integer, primary key
course_id, integer, foreign key to course, required
teacher_id, integer, foreign key to teacher, required
group_id, integer, foreign key to study_group, required
academic_year, varchar(9), required, for example 2026/2027
semester, smallint, required, value 1 or 2
the combination of course_id, group_id, academic_year and semester is unique, because the same course cannot be planned twice for the same group in the same semester

enrollment

enrollment_id, integer, primary key
student_id, integer, foreign key to student, required
offering_id, integer, foreign key to course_offering, required
enrolled_on, date, required
total_points, numeric(5,2), value from 0 to 100
letter_grade, varchar(2)
the combination of student_id and offering_id is unique, so one student cannot be registered twice for the same taught course

Integer identifiers are used as primary keys everywhere, because they are short, never change and are convenient for foreign keys. Natural values such as the code of a group or the e-mail address are kept as unique attributes.

Task 3. Relationships, cardinality and participation

department and teacher, one-to-many. A teacher works in exactly one department, so the participation of teacher is total; a department may temporarily have no teachers, so its participation is partial. Implemented by teacher.department_id.

department and course, one-to-many. Every course belongs to one department (total participation of course), a department may have no courses yet (partial). Implemented by course.department_id.

department and study_group, one-to-many. A group is formed by exactly one department (total), a department may have no groups (partial). Implemented by study_group.department_id.

study_group and student, one-to-many. A student belongs to exactly one group (total), a group that has just been created may still be empty (partial). Implemented by student.group_id.

course and course_offering, one-to-many. An offering always refers to one course (total), a course may not be taught in the current semester (partial). Implemented by course_offering.course_id.

teacher and course_offering, one-to-many. Each offering has exactly one teacher (total), a teacher may have no load in a semester (partial). Implemented by course_offering.teacher_id.

study_group and course_offering, one-to-many. Each offering is planned for exactly one group (total), a group may have no planned courses yet (partial). Implemented by course_offering.group_id.

student and enrollment, one-to-many. A registration always belongs to one student (total), a student may have no registrations (partial). Implemented by enrollment.student_id.

course_offering and enrollment, one-to-many. A registration always refers to one offering (total), an offering may have no registered students yet (partial). Implemented by enrollment.offering_id.

The only many-to-many relationship of the subject area is between student and course_offering. It is resolved by the enrollment entity, which replaces it with two one-to-many relationships and additionally stores the date of the registration and the result.

Task 4. Entity boxes

The diagram was drawn in draw.io (diagrams.net). Every entity is placed in a separate rectangle, and the name of the entity is written in the coloured header of the box. The boxes are arranged from the general to the particular: department is at the top, teacher, course and study_group are below it, then course_offering, student and enrollment.

Task 5. Attributes inside the boxes

All the attributes of an entity are listed inside its box, one attribute per line, in the format name and data type. The primary key is written first and is underlined and marked PK. The foreign keys follow it and are marked FK, so it is possible to see from which table each value comes. The remaining descriptive attributes are placed below the keys.

Task 6. Relationship lines

The entities are connected by lines that go from the primary key of the parent table to the foreign key of the child table. Every line is signed with the name of the relationship, for example employs, owns, teaches, consists of. At both ends of the line the cardinality is shown by the shape of the end and is duplicated by the label (min, max), so that the constraint can be read for each side separately.

Task 7. Legend and explanation of the notation

The diagram uses crow's foot notation. The symbols mean the following:

- a vertical bar means exactly one, label (1,1);
- a crow's foot with a bar means one or many, label (1,N);
- a crow's foot with a circle means zero or many, label (0,N);
- a circle means that the participation is optional, a bar means that it is obligatory.

The symbol and the label are always read for the entity that the end of the line touches. For example, on the line between study_group and student the end near study_group is a bar (1,1) and the end near student is a circle with a crow's foot (0,N). It is read in two directions: one student belongs to exactly one group, and one group contains zero or many students.

Inside the boxes, PK marks the primary key that uniquely identifies a record, FK marks the foreign key that refers to the primary key of another table, and the data type after the name of the attribute shows how the value is stored. Attributes marked as required cannot be empty, and attributes marked as unique cannot repeat in the table..

ERD Diagram for University Student Records Database

